Simulazione dell'algoritmo di Grover con Qiskit

Mini relazione sul file `computazionegrover.py`

1 Obiettivo del programma

Il file `computazionegrover.py` realizza una simulazione dell'algoritmo di Grover, utilizzato per cercare un elemento in uno spazio di ricerca non strutturato. Il circuito considera otto qubit di ricerca, corrispondenti a $N = 2^8 = 256$ possibili stringhe binarie, e amplifica la probabilità di misurare lo stato desiderato

$$|w\rangle = |11111111\rangle.$$

Il target è codificato direttamente nell'oracolo: il programma costituisce quindi un esempio dimostrativo con una sola soluzione nota in fase di costruzione del circuito.

2 Registri e inizializzazione

Il codice utilizza `QuantumCircuit`, `QuantumRegister` e `ClassicalRegister` di Qiskit, insieme al simulatore `AerSimulator` di Qiskit Aer. Vengono creati un registro quantistico di nove qubit e un registro classico di otto bit. I qubit da q_0 a q_7 costituiscono il registro di ricerca; q_8 è un qubit ausiliario, detto *ancilla*, che non viene misurato.

```
qc.h(q[0:8])
qc.x(q[8])
qc.h(q[8])
```

Le porte Hadamard preparano il registro di ricerca nella sovrapposizione uniforme

$$|s\rangle = \frac{1}{16}\sum_{x=0}^{255}|x\rangle,$$

mentre le porte X e H preparano l'ancilla nello stato $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. Inizialmente ogni stringa ha probabilità $1/256$ di essere osservata.

3 Oracolo e diffusore

Il ciclo principale esegue dodici iterazioni, ciascuna composta da un oracolo e da un diffusore.

Oracolo. L'istruzione `qc.mcx(q[0:8], q[8])` applica una porta X all'ancilla soltanto quando tutti gli otto qubit di controllo valgono uno. Poiché $X|-\rangle = -|-\rangle$, questa operazione cambia il segno dell'ampiezza del solo stato cercato, tramite il fenomeno del *phase kickback*. Sul registro di ricerca equivale a

$$O_w = I - 2|w\rangle\langle w|.$$

L'oracolo modifica le fasi, ma da solo non modifica le probabilità di misurazione.

Diffusore. La sequenza successiva applica porte H e X a tutti i qubit di ricerca, riutilizza la stessa porta multicontrollata e infine applica nuovamente X e H. Il diffusore standard di Grover è

$$D = 2|s\rangle\langle s| - I,$$

e trasforma ciascuna ampiezza a_x in $2\bar{a} - a_x$, dove $\bar{a}$ è la media delle ampiezze. La sequenza implementata nel file realizza $I - 2|s\rangle\langle s| = -D$: la differenza è soltanto una fase globale, che non cambia le probabilità osservabili. L'ancilla rimane nello stato $|-\rangle$ e può essere riutilizzata a ogni iterazione.

4 Numero di iterazioni e risultato atteso

Per una singola soluzione, il numero di iterazioni ottimale è vicino a

$$r \simeq \frac{\pi}{4}\sqrt{N}.$$

Più precisamente, ponendo $\theta = \arcsin(1/\sqrt{N})$, si sceglie un intero vicino a $\pi/(4\theta) - 1/2$. Per $N = 256$ si ottiene $r = 12$, come indicato nel codice. La probabilità teorica di successo dopo r iterazioni è

$$P_r = \sin^2\big((2r+1)\theta\big), \qquad P_{12} \approx 0{,}999947.$$

Un numero maggiore di iterazioni non garantisce un risultato migliore: la probabilità oscilla e può diminuire dopo il massimo.

5 Simulazione e misurazione

Il circuito viene simulato con `AerSimulator(method="statevector")` e misurato per `shots = 1024` esecuzioni. Le misure dei primi otto qubit vengono salvate nei corrispondenti bit classici; il metodo `get_counts` restituisce un dizionario che associa ogni stringa osservata al suo numero di occorrenze.

Nel modello ideale utilizzato, il conteggio medio atteso per `11111111` è circa $1024P_{12} = 1023{,}95$. È quindi plausibile ottenere tutte le 1024 misure sul target, ma non è garantito: il campionamento può produrre occasionalmente altri risultati. Questi valori sono previsioni teoriche, non l'esito di un'esecuzione del file. Il codice non specifica un modello di rumore né un seme casuale per rendere ripetibile il campionamento.

6 Conclusioni

Il programma mostra come l'alternanza tra inversione di fase del target e diffusione concentri quasi interamente la probabilità sullo stato cercato. In generale Grover richiede $O(\sqrt{N})$ interrogazioni all'oracolo, contro $O(N)$ di una ricerca classica non strutturata. Questo vantaggio riguarda il numero di interrogazioni nel modello quantistico: non implica che una simulazione classica con Qiskit sia più veloce della ricerca classica. Nell'esempio, inoltre, la porta multicontrollata è descritta ad alto livello; la sua realizzazione su hardware richiederebbe una decomposizione in porte native.